

DOCUMENTO DE INGENIERÍA & ARQUITECTURA

Manual Técnico de Arquitectura, Configuración y Mantenimiento

Sistema de Entrega y Distribución Alimentaria (SEDA) — SEDA v1.0.0

Integrantes del Equipo:	- Martinez Morales David - Lopez Beristain Daniel - Garcia Sanchez Nestor Antoine - Medina Cobos Elian - Bazan Aguila Maria Jose - Bañuelos Cano Carlos Gabriel
Grupo Académico:	9A
Organización:	SEDA
Entorno de Persistencia:	SQLite3 Relacional (Modo WAL habilitado)
Frameworks Utilizados:	React 18 / Vite 8 + Node.js 18 (Express)

Tabla de Contenido Oficial

1. Objetivos del Sistema y Alcance de Ingeniería	3
2. Normas, Políticas de Inocuidad y Seguridad Informática	3
3. Guía de Preparación del Entorno de Desarrollo (Paso a Paso)	4
4. Arquitectura de Software e Integración de Capas	5
5. Estructura Completa del Repositorio del Proyecto	6
6. Diseño de Base de Datos y Diccionario de Datos Exhaustivo	7
7. Especificación del API REST e Interfaces de Usuario	8
8. Plan de Mantenimiento Operativo y Gestión de Backups	9

1. Objetivos del Sistema y Alcance de Ingeniería

El **Sistema de Entrega y Distribución Alimentaria (SEDA)** es una plataforma diseñada para automatizar y optimizar las operaciones críticas de gestión y reparto. El sistema mitiga los problemas de pérdida de alimentos ocasionados por la falta de clasificación oportuna de excedentes procedentes de cadenas minoristas de autoservicio (Walmart, Costco, Sam's) y elimina la inequidad en la distribución comunitaria.

1.1. Arquitectura de Requerimientos Clave

- **Trazabilidad de Pesaje:** Control exacto de ingresos de mercancía medidas en kilogramos directamente vinculados al donante correspondiente.
- **Prevención Dinámica de Mermas:** Flujo de estados configurado para rastrear y priorizar el procesamiento rápido de insumos lácteos y perecederos según sus rangos de temperatura.
- **Reparto Inteligente y Priorizado:** Un algoritmo que determina automáticamente la distribución hacia 20 comunidades registradas, ordenando la logística en base a niveles críticos de necesidad socioeconómica (clases "Muy Malo" y "Regular").

2. Normas, Políticas de Inocuidad y Seguridad Informática

La operación física y lógica de la plataforma SEDA debe cumplir rigurosamente con los estándares establecidos por la Dirección de IT y Control de Calidad Alimentaria:

Norma Técnica de Inocuidad Térmica (Lácteos)

Cualquier lote ingresado al almacén clasificado en la categoría "**Lácteos**" requiere el registro obligatorio de temperatura. Si la temperatura ingresada excede los **4.0°C**, el backend marcará automáticamente el lote en un estado de inspección restrictivo y bloqueará su asignación a rutas de empaque masivo para evitar la rotura de la cadena de frío y mitigar riesgos sanitarios.

2.1. Políticas de Seguridad de Datos y Control de Acceso (RBAC)

- **Control de Acceso Basado en Roles (RBAC):** Las credenciales de acceso se asocian estrictamente a cuatro perfiles operativos:
 - *Administrador:* Acceso total, generación de reportes e impacto mensual, configuración del sistema.
 - *Recepción:* Gestión de ingresos de donaciones, pesaje y control térmico inicial.
 - *Empacador:* Gestión del inventario en almacén y armado de despensas en base a caducidades.
 - *Repartidor:* Vista móvil adaptada para el seguimiento del estado de rutas asignadas.
- **Cifrado y Autenticación:** Todas las contraseñas se almacenan procesadas mediante hash criptográfico bcrypt con un factor de trabajo de 10 rounds. Las llamadas al backend requieren un JSON Web Token (JWT) válido en el header de autorización HTTP.

3. Guía de Preparación del Entorno de Desarrollo (Paso a Paso)

Para configurar un nodo local de desarrollo de forma óptima y estandarizada entre los desarrolladores del equipo, siga estrictamente esta secuencia técnica:

3.1. Requisitos de Software del Sistema

- **Node.js:** Versión v18.16.0 o superior (Se recomienda el uso de NVM para evitar conflictos de runtime).
- **Herramientas de compilación de SQLite:** Para Windows, se requiere instalar las herramientas de compilación de C++ para compilar los binarios nativos del driver `sqlite3`, o en su defecto, instalar a través de `chocolatey` o `npm` con soporte de pre-construidos.
- **Herramientas de CLI de Git:** Para el control de versiones y clonación del repositorio.

3.2. Paso 1: Clonación del Repositorio e Instalación Global de Dependencias

Abra una terminal en su máquina de desarrollo local y ejecute secuencialmente los siguientes comandos:

```
# Clonar el repositorio principal del proyecto
git clone https://github.com/tu-organizacion/seda-platform.git
cd SEDA/Metodologia/Artefactos/seda_appWeb

# Instalar las dependencias de producción y desarrollo comunes para el frontend
npm install
```

3.3. Paso 2: Instalación de Dependencias del Servidor de Backend

El backend de Node.js reside en su propio subdirectorio aislado. Acceda a él e instale los módulos requeridos:

```
# Navegar al directorio del backend
cd server

# Instalar dependencias requeridas (Express, SQLite3, Bcrypt, JWT)
npm install
```

3.4. Paso 3: Configuración Detallada del Archivo de Variables de Entorno

Cree un archivo llamado `.env` en el directorio raíz del backend (`/server/.env`) utilizando el siguiente esquema y adapte los valores según el entorno:

```
# Puerto donde correrá el servidor de Express
PORT=3000

# Ruta absoluta o relativa al archivo físico sqlite
DB_PATH=./data/seda.db

# Clave secreta fuerte para el firmado de tokens de sesión JWT
JWT_SECRET=SD_476_Comidita_Secret_Token_2026

# Configuración de entorno (development | production)
NODE_ENV=development
```

```
# Orígenes permitidos para peticiones CORS (Frontend)
ALLOWED_ORIGIN=http://localhost:5173
```

3.5. Paso 4: Inicialización y Migración de Base de Datos

Para levantar la base de datos por primera vez con el esquema físico inicial de tablas y registros base de prueba (Seeders), ejecute el script de automatización:

```
# Ejecutar migración e inicialización de BD
npm run db:setup

# (O alternativamente mediante el script nativo de node)
node src/config/dbInit.js
```

3.6. Paso 5: Ejecución de Servidores en Modo Desarrollo

Para arrancar ambos entornos con recarga automática activa ante cambios de código:

```
# Para iniciar el backend (en la carpeta /server)
npm run dev

# En otra terminal, para arrancar el servidor de desarrollo de Vite (carpeta raíz /seda_appWeb)
npm run dev
```

4. Arquitectura de Software e Integración de Capas

La plataforma SEDA adopta un modelo de arquitectura de tres capas desacoplada. Esta separación física de responsabilidades garantiza que las modificaciones visuales en la interfaz de React no afecten directamente el motor de base de datos relacional y viceversa.

ESQUEMA TÉCNICO DE INTERACCIÓN DE CAPAS SEDA

```
+-----+
| CAPA DE PRESENTACIÓN (React v18 SPA + TailwindCSS + Vite) |
| - Zustand: Manejo de estado global en memoria de cliente |
| - Axios: Cliente HTTP con Interceptores para inyección de JWT|
+-----+
|                                     | (Peticiónes HTTPS / JSON)
|                                     v
+-----+
| CAPA DE LÓGICA (Node.js runtime + Express Server Framework) |
| - Middlewares: Auth (JWT Validation), Cors, Helmet |
| - Controladores: Lógica de Negocio y Validación de Negocio |
+-----+
|                                     | (Acceso Nativo SQL a través de SQLite3)
|                                     v
+-----+
| CAPA DE PERSISTENCIA (SQLite3 Base de Datos Relacional) |
| - seda.db: Archivo local con bloqueo a nivel de escritura |
| - Pragma: foreign_keys = ON & Journal Mode = WAL |
+-----+
```

4.1. Configuración de Proxy en el Servidor de Desarrollo (Vite)

Para evitar problemas con políticas de orígenes cruzados (CORS) durante el desarrollo local, el archivo `vite.config.js` en el frontend debe configurarse para enrutar las peticiones al puerto del backend automáticamente:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:3000',
        changeOrigin: true,
        secure: false,
      }
    }
  }
});
```

4.2. Middleware de Validación de Roles (Backend)

La seguridad dentro del ecosistema se refuerza a través de un middleware de control de accesos. Ejemplo de implementación del validador de roles en Express:

```
const authorizeRoles = (...allowedRoles) => {  
  return (req, res, next) => {  
    if (!req.user || !allowedRoles.includes(req.user.role)) {  
      return res.status(403).json({  
        error: "Acceso denegado: permisos insuficientes para realizar esta acción."  
      });  
    }  
    next();  
  };  
};
```

5. Estructura Completa del Repositorio del Proyecto

A continuación se define la jerarquía formal de carpetas y archivos esenciales para el despliegue del proyecto. El cumplimiento estricto de este orden de directorios evita fallas en las rutas relativas de carga de dependencias:

ESTRUCTURA DEL BACKEND (/SERVER)

```
server/
├── data/
│   └── seda.db (Persistencia de base de datos)
├── src/
│   ├── config/
│   │   ├── dbInit.js (Inicializador de BD y seeders)
│   │   └── database.js (Conexión sqlite3 activa)
│   ├── controllers/
│   │   ├── authController.js (Login, registro de usuarios)
│   │   ├── donationController.js (Pesaje y temperatura)
│   │   └── routeController.js (Rutas logísticas de reparto)
│   ├── middleware/
│   │   ├── authMiddleware.js (Verificación de token JWT)
│   │   └── roleMiddleware.js (Validación RBAC)
│   ├── routes/
│   │   ├── index.js (Concentrador de enrutamiento)
│   │   ├── auth.routes.js
│   │   ├── donations.routes.js
│   │   └── routes.routes.js
│   └── index.js (Archivo de arranque principal del API)
├── package.json (Configuración de Express y dependencias)
└── .env (Configuración de puertos y secretos del sistema)
```

ESTRUCTURA DEL FRONTEND (/SRC)

```
src/
├── assets/ (Logotipos y recursos gráficos est)
├── components/
│   ├── common/ (Botones, inputs, modales gene)
│   └── layout/ (Estructura de sidebar y navba)
├── context/
│   └── useStore.ts (Manejo de estado global c)
├── views/
│   ├── Logins/
│   │   ├── LoginContainer.tsx
│   │   └── LoginView3.tsx
│   ├── Dashboard.tsx (Panel de Control de IT)
│   ├── Packaging.tsx (Mapeo de empaque y desc)
│   └── Logistics.tsx (Monitoreo de rutas prio)
├── App.tsx (Estructura de enrutamiento princi)
├── main.tsx (Punto de anclaje para el DOM de)
├── index.css (Configuración e importación de)
├── package.json (Configuración de compilación)
└── vite.config.js (Configuración de empaqueta)
```

5.1. Manejo de Estado en Cliente con Zustand

Para evitar el sobrediseño con arquitecturas complejas como Redux, el sistema SEDA implementa **Zustand**, un gestor de estados sumamente ligero que permite la sincronización reactiva del usuario logueado en todas las vistas de la aplicación:

```
import { create } from 'zustand';

interface User {
  username: string;
  role: 'Admin' | 'Recepción' | 'Empacador' | 'Repartidor';
}

interface AuthState {
  user: User | null;
  token: string | null;
  setLogin: (user: User, token: string) => void;
  logout: () => void;
}

export const useStore = create<AuthState>((set) => ({
  user: null,
  token: null,
  setLogin: (user, token) => set({ user, token }),
  logout: () => set({ user: null, token: null }),
})));
```

6. Diseño de Base de Datos y Diccionario de Datos Exhaustivo

El motor de base de datos relacional de SQLite garantiza transacciones robustas a nivel local. Para un óptimo rendimiento en despliegues con alta lectura, el conector debe forzar el modo de diario **WAL (Write-Ahead Logging)** y la validación de claves foráneas.

6.1. Código de Inicialización Física de SQLite3 (dbInit.js)

Este script define de manera estructurada el esquema físico de base de datos relacional al levantar el servidor:

```
const sqlite3 = require('sqlite3').verbose();
const db = new sqlite3.Database('./data/seda.db');

db.serialize(() => {
  db.run("PRAGMA foreign_keys = ON;");
  db.run("PRAGMA journal_mode = WAL;");

  // Creación de Tabla de Usuarios
  db.run(`CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL CHECK(role IN ('Admin', 'Recepción', 'Empacador', 'Repartidor'))
  );`);

  // Creación de Tabla de Donantes
  db.run(`CREATE TABLE IF NOT EXISTS donors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL UNIQUE
  );`);

  // Creación de Tabla de Donaciones (Inventario)
  db.run(`CREATE TABLE IF NOT EXISTS donations (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    donor_id INTEGER NOT NULL,
    category TEXT NOT NULL CHECK(category IN ('Lácteos', 'Enlatados', 'Frutas y Verduras', 'Secos')),
    weight_kg REAL NOT NULL CHECK(weight_kg > 0),
    temperature_c REAL DEFAULT NULL,
    date_registered DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY(donor_id) REFERENCES donors(id) ON DELETE RESTRICT
  );`);

  // Creación de Tabla de Comunidades
  db.run(`CREATE TABLE IF NOT EXISTS communities (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL UNIQUE,
    vulnerability_level TEXT NOT NULL CHECK(vulnerability_level IN ('Muy Malo', 'Regular'))
  );`);
});
```

6.2. Detalle de Diccionario de Datos del Almacén

Tabla: donations (Inventario de Materia Prima)

Nombre del Campo	Tipo SQLite	Restricciones	Descripción / Regla de Negocio
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Clave primaria correlativa única para cada lote de alimentos.
donor_id	INTEGER	FOREIGN KEY NOT NULL	Identificador referencial de la tabla de donadores (donors).
category	TEXT	NOT NULL (CHECK)	Clasificación de mermas: 'Lácteos', 'Enlatados', 'Frutas y Verduras', 'Secos'.
weight_kg	REAL	NOT NULL (CHECK > 0)	Peso neto del lote ingresado al almacén, expresado en kilogramos.
temperature_c	REAL	DEFAULT NULL	Obligatorio para Lácteos. Si es > 4.0°C, se restringe la salida del lote.
date_registered	DATETIME	DEFAULT CURRENT_TIMESTAMP	Sello de tiempo automático generado en el servidor de base de datos.

7. Especificación del API REST e Interfaces de Usuario

El intercambio de datos entre la interfaz visual construida con React y el motor lógico basado en Express.js se realiza estrictamente a través de transacciones HTTP REST con formatos de datos codificados en JSON.

7.1. Tabla de Endpoints Críticos de SEDA

Método	Ruta del Endpoint	Acceso (RBAC)	Estructura JSON de Entrada / Salida
POST	/api/auth/login	Público	In: {"username": "usr", "password": "123"} Out (200): {"token": "JWT_TOKEN", "role": "Admin"}
GET	/api/donations	Recepción, Admin	In: Vacío (Headers con token) Out (200): Lista completa de lotes registrados con sus pesos y temperaturas.
POST	/api/donations	Recepción, Admin	In: {"donor_id": 1, "category": "Lácteos", "weight_kg": 150, "temperature_c": 3.4} Out (201): Confirmación del lote creado.
GET	/api/routes/priority	Repartidor, Admin	In: Vacío Out (200): Lista de las 20 comunidades ordenadas algorítmicamente por nivel de vulnerabilidad.

7.2. Implementación de Alertas Visuales en el Tablero de Control

En el Frontend de React, la alerta de inocuidad térmica para lácteos se renderiza de manera reactiva mapeando el arreglo de donaciones activas:

```
import React from 'react';

interface Donation {
  id: number;
  category: string;
  temperature_c: number | null;
  weight_kg: number;
}

export const InocuidadAlertList: React.FC<{ donations: Donation[] }> = ({ donations }) => {
  return (
    <div className="space-y-2">
      {donations.map((item) => {
        const isCriticalColdBroken = item.category === 'Lácteos' && item.temperature_c !== null && item.temperature_c > 3.5;
        return (
          <div key={item.id} className={`p-3 border rounded ${isCriticalColdBroken ? 'bg-red-50 border-red-500' : 'bg-white border-gray-200'}`>
            <span className="font-bold">ID Lote: #{item.id}</span> - Categoría: {item.category} ({item.weight_kg} kg)
            {isCriticalColdBroken && (
              <p className="text-xs font-bold mt-1">
                ⚠️ ALERTA: Cadena de frío rota ({item.temperature_c}°C). Lote bloqueado para entrega.
              </p>
            )}
          </div>
        );
      })}
    </div>
  );
};
```

```
</div>  
);  
};
```

8. Plan de Mantenimiento Operativo y Gestión de Backups

La alta disponibilidad del sistema SEDA depende de la correcta ejecución de tareas programadas de mantenimiento y respaldos a nivel de infraestructura:

8.1. Estrategia de Respaldos de SQLite en Caliente

Para evitar colisiones de escritura con usuarios que se encuentren registrando pesaje activamente, los administradores de IT deben programar un script diario utilizando el comando nativo de respaldo en caliente de sqlite:

```
#!/bin/bash
# Script de Respaldo Seguro - SEDA v1.0.0
DB_FILE="/var/www/seda/server/data/seda.db"
BACKUP_DIR="/var/backups/seda"
DATE_STAMP=$(date +%Y%m%d_%H%M%S")

# Garantizar el directorio de destino
mkdir -p "$BACKUP_DIR"

# Ejecución de respaldo en caliente
sqlite3 "$DB_FILE" ".backup '$BACKUP_DIR/seda_backup_${DATE_STAMP}.db'"

# Compresión del respaldo para ahorrar espacio en disco
gzip "$BACKUP_DIR/seda_backup_${DATE_STAMP}.db"

# Retención de respaldos de los últimos 15 días (limpieza automática)
find "$BACKUP_DIR" -name "seda_backup_*.gz" -mtime +15 -exec rm {} \;
```

8.2. Mantenimiento Físico y Desempeño Semestral

Debido al comportamiento dinámico del flujo de estados Kanban (constante creación y eliminación lógica de lotes), el archivo físico `seda.db` puede fragmentarse acumulando páginas de datos vacías. El administrador técnico debe programar cada seis meses un script para ejecutar las siguientes directivas de optimización estructural:

```
# Optimización Semestral de Base de Datos
sqlite3 ./server/data/seda.db "VACUUM; REINDEX; ANALYZE;"
```

- **VACUUM:** Reconstruye la base de datos por completo reduciendo el peso en disco del archivo al mínimo indispensable.
- **REINDEX:** Reconstruye todos los índices de búsqueda rápida optimizando la consulta de endpoints.
- **ANALYZE:** Recopila estadísticas sobre las tablas e índices para que el planificador de consultas de SQLite tome decisiones de ejecución más veloces.